

YISAKAL SACCO DATABASE NORMALIZATION ANALYSIS

(A step by step normalization to Boyce-Codd Normal Form (BCNF))

This normalization analysis document presents how the data is modeled and is normalized by considering the structural entities and relationships defined in the conceptual ERD. It follows series of steps followed to obtain a database design that allows for consistent storage and efficient access of data in a relational database. These steps reduce data redundancy and the risk of data becoming inconsistent.

- In order to obtain a normalized form to BCNF, we must follow the following steps:-
 - **Step 1: Create Unnormalized Data:-** Construct a wide, un-atomic, messy relation combining highly integrated metrics to simulate raw application states and serve as the baseline for structurally organizing it.
 - **Step 2: Convert to 1st Normal Form (1NF):-** Eliminate repeating data collections, cross-product records, and multi-valued string lists. It enforces strict structural atomicity.
 - **Step 3:- Convert to 2nd Normal Form (2NF):-** Identify and remove all partial dependencies. Ensure that every non-prime attribute exhibits full functional dependency on the designated primary key framework.
 - **Step 4:- Convert to 3rd Normal Form (3NF):-** Identify and isolate all transitive dependency chains. Ensure that no non-prime structural element relies functionally on another non-prime attribute.
 - **Step 5:- Convert to Boyce-Codd Normal Form (BCNF):** Perform final determinant verification. Ensure that for every non-trivial functional dependency $X \rightarrow Y$, the left-hand side determinant X constitutes a valid, indisputable super key or candidate key.

Part 1: Administrative location, Branch and Member Subsystem

Step 1:- Create unnormalized data

The unnormalized relation consists customer demographics, variable multi-valued contact entries, geographical elements, and operational parent branch office definitions inside a single flat table grid.

member_id	full_name	branch_id	national_id	phone_number	Kebele	subcity	city	region	branch_name
M-901	Abebe Kebede Balcha	BR-001	NID-A9901	+25191111, +25192222	03	Bole	A.A	A.A	Bole branch
M-902	Chaltu Hunde Tafa	BR-001	NID-O2241	+25191144	01	Sebeta	Sebeta	Oromia	Sebeta branch

Step 2:- Convert to 1NF:-

The row attributes display singular structural atomicity. Multi-valued lists are avoided since individual member and branch phone contact records are maintained within single-value attributes matching the operational system layout..

member_id	full_name	branch_id	national_id	phone_number	Kebele	subcity	city	region	branch_name
M-901	Abebe Kebede Balcha	BR-001	NID-A9901	+25191111,	03	Bole	A.A	A.A	Bole branch
M-901	Abebe Kebede Balcha	BR-001	NID-A9901	+25192222	03	Bole	A.A	A.A	Bole branch
M-902	Chaltu Hunde Tafa	BR-001	NID-O2241	+25191144	01	Sebeta	Sebeta	Oromia	Sebeta branch

Initial Composite Primary Key: (member_id, branch_id)

FD 1.1: (member_id, branch_id) → full_name, national_id, kebele, subcity, city, region, phone_number, branch_name (**Full Key Dependency**)

FD 1.2: branch_id → branch_name, region, city, subcity, kebele (**Partial Key Dependency**)

FD 1.3: member_id → full_name, branch_id, national_id, region, city, subcity, kebele, phone_number, (**Alternative Unique Natural Key**)

Determinant)

Step 3:- Convert to 2NF:-

We isolate attributes that suffer from partial dependency on the subset identifier member_id away from the multivalued phone number string tracker.

- **2NF_BRANCH** (branch_id, branch_name, region, city, subcity, kebele)
- **2NF_MEMBER** (member_id, full_name, branch_id, national_id, kebele, phone_number, region, city, subcity)

STEP 4 — Convert to 3NF

Reviewing the schemas reveals that geographical attributes are functionally dependent directly on the specific entity IDs (member_id and branch_id) as explicit attributes of their physical definitions within this specific operational requirement. Transitive dependency analysis checks pass cleanly for both relations.

- **3NF_BRANCH** (branch_id, branch_name, region, city, subcity, kebele)
- **3NF_MEMBER** (member_id, full_name, branch_id, national_id, kebele, phone_number, region, city, subcity)

STEP 5 — Convert to BCNF

We audit the remaining functional determinants. In 3NF_MEMBER, member_id, national_id_number, and kebele_id_number serve as valid candidate keys. In 3NF_BRANCH, branch_id is the absolute determinant. Every determinant matches a superkey profile exactly.

- **Branch** (branch_id, branch_name, region, city, subcity, kebele)
- **Member** (member_id, full_name, branch_id, gender, date_of_birth, national_id, phone_number, region, city, subcity, kebele,)

Part 2: Internal Human Resources and Audit log framework

STEP 1 — Create UNNORMALIZED DATA (Employee-Audit table)

This relation links employee identities, functional system access security roles, and specific transactional operations historical metadata sequences into a single table.

employee_id	employee_full_name	branch_id	phone_number	audit_id	action_type	timestamp	old_values	new_values
E-01	Almaz Tekle Haile	BR-001	+25191111,	AUD-901	APPROVE_LOAN	2026-05-20 10:15	{"status": "Pending"}	{"status": "Approved"}
E-02	Almaz Tekle Balcha	BR-001	+25192222	AUD-942	UPDATE_MEMBER	2026-05-20 11:30	{"house_no": "4555"}	{"house_no": "6555"}
E-03	Bekele Zewdu Assefa	BR-001	+25191114	AUD-955	POST_DEPOSIT	2026-05-20 11:45	{"balance": "20,500"}	{"balance": "26,780"}

STEP 2 — Convert to 1NF

The row attributes display singular structural atomicity. No multivalued data elements exist inside individual cells.

Initial Composite Primary Key: (employee_id, audit_id)

FD 2.1: (employee_id, audit_id) → employeefull_name, branch_id, action_type, timestamp, old_values, new_values (**Full Key Dependency**)

FD 2.2: employee_id → employee_name, branch_id (**Partial Key Dependency Violation**)

FD 2.3: audit_id → employee_id, action_type, timestamp, old_values, new_values (**Partial Key Dependency Violation**)

STEP 3 — Convert to 2NF

Because **employee_name** and **action_type** rely entirely on independent component fractions of the initial key string, we break the relation down to satisfy 2NF compliance rules.

2NF_EMPLOYEE (employee_id, employee_name, branch_id)

2NF_AUDIT_LOG (audit_id, employee_id, action_type, timestamp, old_values, new_values)

STEP 4 — Convert to 3NF

Reviewing the tables reveals that no non-key fields determine other non-key values. Therefore it passes transitive dependency analysis for both relations.

STEP 5 — Convert to BCNF

Every functional dependency determinant matches an explicitly assigned unique table primary key. No further functional structural transformation is required.

Employee (employee_id, employee_name, role, branch_id)

Audit_Log (audit_id, employee_id, action_type, timestamp, old_values, new_values)

Part 3: Financial Products, Saving Accounts and Transactions

Step 1:- Create unnormalized data

This table has interest products configuration constraints, customer savings values, and linear sequential account cash transaction histories into an unseparated data table format.

account_id	account_no	member_id	product_id	product_name	interest_rate	transaction_id	amount	Balance_after_transaction
ACC-501	100023455	M-901	PRD-SAV	Deposit	5.00	TX-8801, TX-8802	5000	25500.00
ACC-502	100023444	M-902	PRD-SAV	Deposit	5.00	TX-8803	7000	34500.00

STEP 2: Convert to 1st Normal Form (1NF)

Every attribute cell holds strictly atomic data. The dynamic collection of attributes is controlled by a composite primary key layout.

- **Primary Key:** (account_id, product_id, transaction_id)
- **Functional Dependencies (FDs):**
 - **FD 3.1 (Composite Key Link):** (account_id, product_id, transaction_id) → amount, balance_after_transaction, account_number, interest_rate
 - **FD 3.2 (Account Cluster):** account_id → account_number, product_id, member_id, branch_id (*Partial Key Dependency Violation*)
 - **FD 3.3 (Product Settings):** product_id → product_name, interest_rate (*Partial Key Dependency Violation*)
 - **FD 3.4 (Transaction Event):** transaction_id → account_id, amount, balance_after_transaction, (*Partial Key Dependency Violation*)

STEP 3: Convert to 2nd Normal Form (2NF)

We remove the partial dependencies identified in Step 2 by isolating attributes that depend on only a subset of the composite key into independent tables.

- **2NF_SAVING_PRODUCT** (product_id, product_name, interest_rate)
- **2NF_SAVING_ACCOUNT** (account_id, account_number, product_id, member_id, branch_id)
- **2NF_SAVINGS_TRANSACTION** (transaction_id, account_id, amount, balance_after_transaction)

STEP 4: Convert to 3rd Normal Form (3NF)

We check for transitive dependencies where a non-prime attribute determines another non-prime attribute (X → Y → Z). Within these isolated boundaries, no transitively linked elements exist; settings rely directly on their respective key roots.

- **3NF_SAVING_PRODUCT** (product_id, product_name, interest_rate)
- **3NF_SAVING_ACCOUNT** (account_id, account_number, product_id, member_id, branch_id)
- **3NF_SAVINGS_TRANSACTION** (transaction_id, account_id, amount, balance_after_transaction)

STEP 5: Convert to Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$; X is a superkey. In 3NF_SAVING_ACCOUNT, account_number is unique and acts as a valid secondary candidate key (account_number $\rightarrow$ account_id...). Because every active left-hand determinant represents a verified superkey, the relations are confirmed in BCNF.

- **SavingAccountProduct** (product_id [PK], product_name, interest_rate)
- **SavingAccount** (account_id [PK], account_number [SK], product_id [FK], member_id, branch_id)
- **SavingsTransaction** (transaction_id [PK], account_id [FK], amount, balance_after_transaction)

Part 4: CAPITAL LOANS, REPAYMENT SCHEDULES, AND ASSET COLLATERAL MAPPING

STEP 1: Create UNNORMALIZED DATA

This table combines active loan commitments, targeted guarantor financial limits, and asset risk records into a single table.

loan_id	principal_amount	schedule_id	installment_number	collateral_id	ownership_document_ref	guarantor_member_id	guarantee_amount
LN-800	500000	SCH-01	01	COL-55	REF-VEH99	M-905	200000
LN-801	750000	SCH-02	02	COL-56	REF-PROP4	M-906	500000

STEP 2: Convert to 1st Normal Form (1NF)

To make the data easier to organize in tables, attributes that can have many values are separated into simpler values. Since each record contains combined information, a larger primary key made from several attributes is used to uniquely identify each row without duplication

- **Primary Key:** (loan_id, schedule_id, collateral_id, guarantor_member_id)
- **Functional Dependencies (FDs):**
 - **FD 4.1 (Loan Master):** loan_id -> principal_amount, product_id, , term_months (*Partial Key Dependency Violation*)
 - **FD 4.2 (Amortization Line):** schedule_id -> loan_id, installment_number, due_date (*Partial Key Dependency Violation*)
 - **FD 4.3 (Secured Collateral):** collateral_id -> loan_id, ownership_document_ref, estimated_value, status (*Partial Key Dependency Violation; maps 1:M to Loan directly per the schema specification*)
 - **Guaranty Matrix):** (loan_id, guarantor_member_id) -> guarantee_amount, status (*Partial Key Dependency Violation*)

STEP 3: Convert to 2nd Normal Form (2NF)

We systematically eliminate the structural partial dependencies verified in Step 2 by isolating each dependent cluster into independent, focused tables.

2NF_LOAN

loan_id	principal_amount	product_id	term_months
LN-880	500000	LP-01	12 Months
LN-881	750000	LP-02	24 Months

2NF_LOAN_SCHEDULE

schedule_id	loan_id	installment_number	due_date
SCH-01	LN-880	01	2026-03-01
SCH-02	LN-881	02	2026-04-15

2NF_COLLATERAL

collateral_id	loan_id	ownership_document_status	estimated_value	status
COL-55	LN-880	REF-VEH99	900000	Pledge
COL-56	LN-881	REF-PROP4	150000	Pledge

2NF_GUARANTY

loan_id	guarantor_member_id	guarantee_amount	status
LN-880	M-905	200000	Active
LN-881	M-906	350000	Active

STEP 4: Convert to 3rd Normal Form (3NF)

We audit the 2NF structures for transitive dependencies ($X \rightarrow Y \rightarrow Z$) where non-prime attributes functionally determine other non-prime elements. Within these four isolated limits, all attributes depend directly on their respective key roots without intermediate links.

- **3NF_LOAN** (Same attributes as 2NF_LOAN)
- **3NF_LOAN_SCHEDULE** (Same attributes as 2NF_LOAN_SCHEDULE)
- **3NF_COLLATERAL** (Same attributes as 2NF_COLLATERAL)
- **3NF_GUARANTY** (Same attributes as 2NF_GUARANTY)

STEP 5: Convert to Boyce-Codd Normal Form (BCNF)

A relation satisfies BCNF if every determinant is a superkey. In 3NF_COLLATERAL, the unique registration reference field forms a secondary candidate key (ownership_document_ref -> collateral_id, loan_id...). Because every non-trivial dependency determinant represents an absolute superkey, BCNF is met cleanly.

- **Loan** (loan_id [PK], principal_amount, product_id, loan_officer_id, term_months)
- **LoanSchedule** (schedule_id [PK], loan_id [FK], installment_number, due_date)
- **Collateral** (collateral_id [PK], loan_id [FK], ownership_document_ref [SK], estimated_value, status)
- **Guaranty** ((loan_id, guarantor_member_id) [PK/FK], guarantee_amount, status)

Part 5: SYSTEM FEE RULES, CHARGING FRAMEWORK, AND EXECUTION INSTANCES

STEP 1: Create UNNORMALIZED DATA

This processing layer captures execution log instances, operational link triggers (loans vs. transactional accounts), and the system configuration properties that calculate fees.

fee_transaction_id	amount	fee_event_id	loan_id	saving_transaction_id	fee_type_id	calculation_method	amount
FT-11	150.00	EV-101	LN-880	TX-8801	FT-PERC	Percentage	1.50
FT-12	700.00	EV-102	LN-881	TX-8899	FT-FLAT	Flat	50.00

STEP 2: Convert to 1st Normal Form (1NF)

All entries are simplified into clear, atomic cell components.

- **Initial Primary Key:** fee_transaction_id
- **Functional Dependencies (FDs):**
 - **FD 5.1:** fee_transaction_id -> fee_event_id, amount, reference, transaction_date
 - **FD 5.2:** fee_event_id -> fee_type_id, loan_id, saving_transaction_id, paid
 - **FD 5.3:** fee_type_id -> name, description, calculation_method, amount_or_rate, is_active (*Transitive Linkage*)

STEP 3: Convert to 2nd Normal Form (2NF)

Because the unnormalized baseline relation relies on a single-column primary key identifier (fee_transaction_id), partial key functional dependencies are mathematically impossible. The table satisfies 2NF parameters natively.

STEP 4: Convert to 3rd Normal Form (3NF)

The schema exhibits transitive dependency chains where the primary identifier dictates a non-prime attribute, which in turn determines other non-prime values (fee_transaction_id -> fee_event_id -> fee_type_id). We resolve these anomalies by decomposing the structure into three distinct tables.

3NF_FEE_TYPE

fee_type_id	name	description	calculation_method	amount_or_rate	is_active
FT-PERC	Processing Fee	Standard	Percentage	1.50	true
FT-FLAT	ATM Withdrawal Fee	Penalty	Flat	50.00	true

3NF_FEE_EVENT

fee_event_id	fee_type_id	loan_id	saving_transaction_id	paid
EV-101	FT-PERC	LN-880	TX-8801	false
EV-102	FT-FLAT	LN-881	TX-8899	true

3NF_FEE_TRANSACTION

fee_transaction_id	fee_event_id	amount	reference	transaction_date
FT-11	EV-101	150.00	REF-FEE01	2026-05-20
FT-12	EV-102	50.00	REF-FEE02	2026-05-21

STEP 5: Convert to Boyce-Codd Normal Form (BCNF)

We audit the remaining functional dependencies. Every active left-hand determinant matches the assigned primary key superkey for its respective table, confirming full compliance with BCNF standards.

- **FeeType** (fee_type_id [PK], name, description, calculation_method, amount_or_rate, is_active)
- **FeeEvent** (fee_event_id [PK], fee_type_id [FK], loan_id, saving_transaction_id, paid)
- **FeeTransaction** (fee_transaction_id [PK], fee_event_id [FK], amount, reference, transaction_date)

Comprehensive Verified Candidate Key Super-Matrix

This verification matrix confirms that every mapped table directly aligns with your physical SQL design while operating cleanly under Boyce-Codd Normal Form (BCNF) requirements.

Target Table Name	Primary Key (PK)	Candidate Keys (CK) / Determinants	BCNF Compliance Status
Branch	branch_id	branch_id	CONFIRMED (Superkey)
Employee	employee_id	employee_id	CONFIRMED (Superkey)
Member	member_id	member_id, national_id_number, kebele_id_number	CONFIRMED (Superkey)
SavingAccountProduct	product_id	product_id	CONFIRMED (Superkey)
LoanProduct	product_id	product_id	CONFIRMED (Superkey)
SavingAccount	account_id	account_id, account_number	CONFIRMED (Superkey)
Loan	loan_id	loan_id	CONFIRMED (Superkey)
LoanSchedule	schedule_id	schedule_id	CONFIRMED (Superkey)
SavingsTransaction	transaction_id	transaction_id	CONFIRMED (Superkey)
LoanTransaction	transaction_id	transaction_id	CONFIRMED (Superkey)
Guaranty	(loan_id, member_id)	(loan_id, member_id)	CONFIRMED (All keys prime)
Collateral	collateral_id	collateral_id, ownership_document_ref	CONFIRMED (Superkey)
Audit	audit_id	audit_id	CONFIRMED (Superkey)
FeeType	fee_type_id	fee_type_id	CONFIRMED (Superkey)
FeeEvent	fee_event_id	fee_event_id	CONFIRMED (Superkey)
FeeTransaction	fee_transaction_id	fee_transaction_id	CONFIRMED (Superkey)

Conclusion

The system design was simplified from a complex raw structure into 16 well-organized tables.

Some data that was scattered or repeated was grouped properly (like location details and collateral information) so it is stored in a cleaner and more efficient way.

The design was also improved step by step by removing redundancy and dependency problems, ensuring each table stores only what it should and depends only on its main key.

In the end, the database became well-structured, consistent, and ready for reliable real-world use without unnecessary duplication or errors.